Assignment 3: Q-Learning and Actor-Critic Algorithms

Due October 18, 11:59 pm

1 Multistep Q-Learning

Consider the N-step variant of Q-learning described in lecture. We learn $Q_{\phi_{k+1}}$ with the following updates:

$$y_{j,t} \leftarrow \left(\sum_{t'=t}^{t+N-1} \gamma^{t'-t} r_{j,t'} \right) + \gamma^N \max_{\mathbf{a}_{j,t+N}} Q_{\phi_k} \left(\mathbf{s}_{j,t+N}, \mathbf{a}_{j,t+N} \right) \tag{1}$$

$$\phi_{k+1} \leftarrow \arg\min_{\phi \in \Phi} \sum_{j,t} \left(y_{j,t} - Q_\phi(\mathbf{s}_{j,t}, \mathbf{a}_{j,t}) \right)^2 \tag{2}$$

In these equations, j indicates an index in the replay buffer of trajectories $\mathcal{D}_k$. We first roll out a batch of B trajectories to update $\mathcal{D}_k$ and compute the target values in (1). We then fit $Q_{\phi_{k+1}}$ to these target values with (2). After estimating $Q_{\phi_{k+1}}$, we can then update the policy through an argmax:

$$\pi_{k+1}(\mathbf{a}_t \mid \mathbf{s}_t) \leftarrow \begin{cases} 1 \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_{\phi_{k+1}}(\mathbf{s}_t, \mathbf{a}_t) \\ 0 \text{ otherwise.} \end{cases} \tag{3}$$

We repeat the steps in eqs. (1) to (3) K times to improve the policy. In this question, you will analyze some properties of this algorithm, which is summarized in Algorithm 1.

Algorithm 1 Multistep Q-Learning

Require: iterations K, batch size B
1: initialize random policy π_0, sample $\phi_0 \sim \Phi$
2: **for** $k = 0 \dots K-1$ **do**
3: Update $\mathcal{D}_{k+1}$ with B new rollouts from π_k
4: compute targets with (1)
5: $Q_{\phi_{k+1}} \leftarrow$ update with (2)
6: $\pi_{k+1} \leftarrow$ update with (3)
7: **end for**
8: **return** π_K

1.1 TD-Learning Bias (2 points)

We say an estimator $f_{\mathcal{D}}$ of f constructed using data $\mathcal{D}$ sampled from process P is *unbiased* when $\mathbb{E}_{\mathcal{D} \sim P}[f_{\mathcal{D}}(x) - f(x)] = 0$ at each x.

Assume $\hat{Q}$ is a noisy (but unbiased) estimate for Q. Is the Bellman backup $\mathcal{B}\hat{Q} = r(s,a) + \gamma \max_{a'} \hat{Q}(s', a')$ an unbiased estimate of $\mathcal{B}Q$?

☐ Yes

☐ No

1.2 Tabular Learning (6 points total)

At each iteration of the algorithm above after the update from eq. (2), Q_{ϕ_k} can be viewed as an estimate of the true optimal Q^*. Consider the following statements:

I. $Q_{\phi_{k+1}}$ is an unbiased estimate of the Q function of the last policy, Q^{π_k}.

II. As $k \to \infty$ for some fixed B, Q_{ϕ_k} is an unbiased estimate of Q^*, i.e., $\lim_{k \to \infty} \mathbb{E}\left[Q_{\phi_k}(s,a) - Q^*(s,a) \right] = 0$.

III. In the limit of infinite iterations and data we recover the optimal Q^*, i.e., $\lim_{k,B \to \infty} \mathbb{E}\left[\| Q_{\phi_k} - Q^* \|_\infty \right] = 0$.

作业 3：Q 学习与 Actor-Critic 算法

截止日期：10月18日，晚上11:59

1 多步 Q 学习

考虑 N 步 在讲座中描述的 Q-learning 变体。我们通过以下更新来学习 $Q_{\phi_{k+1}}$：

$$y_{j,t} \leftarrow \left(\sum_{t'=t}^{t+N-1} \gamma^{t'-t} r_{j,t'} \right) + \gamma^N \max_{\mathbf{a}_{j,t+N}} Q_{\phi_k} \left(\mathbf{s}_{j,t+N}, \mathbf{a}_{j,t+N} \right) \tag{1}$$

$$\phi_{k+1} \leftarrow \arg\min_{\phi \in \Phi} \sum_{j,t} \left(y_{j,t} - Q_\phi(\mathbf{s}_{j,t}, \mathbf{a}_{j,t}) \right)^2 \tag{2}$$

在这些方程中，j 表示回放缓冲区中轨迹 $\mathcal{D}_k$ 的一个索引。我们首先展开一批 B 条轨迹来更新 $\mathcal{D}_k$，并计算式 (1) 中的目标值。然后我们用式 (2) 将 $Q_{\phi_{k+1}}$ 拟合到这些目标值上。在估计出 $Q_{\phi_{k+1}}$ 之后，我们可以通过 argmax 更新策略：

$$\pi_{k+1}(\mathbf{a}_t \mid \mathbf{s}_t) \leftarrow \begin{cases} 1 \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_{\phi_{k+1}}(\mathbf{s}_t, \mathbf{a}_t) \\ 0 \text{ otherwise.} \end{cases} \tag{3}$$

我们将式 (1) 到 (3) 的步骤重复 K 次以改进策略。在本题中，你将分析该算法的一些性质，该算法在算法 1 中进行了总结。

算法 1 多步 Q 学习

要求： 迭代次数 K，批量大小 B
1: 初始化随机策略 π_0，采样 $\phi_0 \sim \Phi$
2: **对于** $k = 0 \dots K-1$ **执行**
3: 使用来自 π_k 的 B 个新采样轨迹更新 $\mathcal{D}_{k+1}$
4: 使用 (1) 计算目标
5: $Q_{\phi_{k+1}} \leftarrow$ 使用 (2) 更新
6: $\pi_{k+1} \leftarrow$ 使用 (3) 更新
7: **结束循环**
8: **返回** π_K

1.1 TD学习偏差（2分）

当在每个 x 上满足 $\mathbb{E}_{\mathcal{D} \sim P}[f_{\mathcal{D}}(x) - f(x)] = 0$ 时，我们称由从过程 P 抽样的数据 $\mathcal{D}$ 构造的估计量 $f_{\mathcal{D}}$ 对 f 是无偏的。

假设 $\hat{Q}$ 是对 Q 的有噪声（但无偏）的估计。Bellman 备份 $\mathcal{B}\hat{Q} = r(s,a) + \gamma \max_{a'} \hat{Q}(s', a')$ 是否为 $\mathcal{B}Q$ 的无偏估计?

☐ 是
☐ 否

1.2 表格学习（6 分）

在上述算法中，在根据等式 (2) 完成更新之后的每次迭代中，Q_{ϕ_k} 可被视为对真实最优 Q^* 的估计。考虑以下陈述：

I. $Q_{\phi_{k+1}}$ 是策略 Q^{π_k}的 Q 函数的无偏估计。**II.** 正如 $k \to \infty$，对于某个固定的 B，Q_{ϕ_k}是 Q^*的无偏估计，即 $\lim_{k \to \infty} \mathbb{E}\left[Q_{\phi_k}(s,a) - Q^*(s,a) \right] = 0$.**III.** 在迭代次数和数据量趋于无限的极限下，我们能恢复最优的 Q^*，即 $\lim_{k,B \to \infty} \mathbb{E}\left[\| Q_{\phi_k} - Q^* \|_\infty \right] = 0$.

We make the additional assumptions:

- The state and action spaces are finite.
- Every batch contains at least one experience for each action taken in each state.
- In the tabular setting, Q_{ϕ_k} can express any function, i.e., $\{Q_{\phi_k} : \phi \in \Phi\} = \mathbb{R}^{S \times A}$.

When updating the buffer $\mathcal{D}_k$ with B new trajectories in line 3 of Algorithm 1, we say:

- When learning *on-policy*, $\mathcal{D}_k$ is set to contain only the set of B new rollouts of π (so $|\mathcal{D}_k| = B$). Thus, we only train on rollouts from the current policy.
- When learning *off-policy*, we use a fixed dataset $\mathcal{D}_k = \mathcal{D}$ of B trajectories from another policy π'.

Indicate which of the statements **I-III** always hold in the following cases. No justification is required.

	I.	**II.**	**III.**
1. $N = 1$ and ...			
(a) on-policy in tabular setting	☐	☐	☐
(b) off-policy in tabular setting	☐	☐	☐
2. $N > 1$ and ...			
(a) on-policy in tabular setting	☐	☐	☐
(b) off-policy in tabular setting	☐	☐	☐
3. In the limit as $N \to \infty$ (no bootstrapping) ...			
(a) on-policy in tabular setting	☐	☐	☐
(b) off-policy in tabular setting	☐	☐	☐

1.3 Variance of Q Estimate (2 points)

Which of the three cases ($N = 1$, $N > 1$, $N \to \infty$) would you expect to have the highest-variance estimate of Q for fixed dataset size B in the limit of infinite iterations k? Lowest-variance?

Highest variance:

☐ $N = 1$
☐ $N > 1$
☐ $N \to \infty$

Lowest variance:

☐ $N = 1$
☐ $N > 1$
☐ $N \to \infty$

1.4 Function Approximation (2 points)

Now say we want to represent Q via function approximation rather than with a tabular representation. Assume that for any deterministic policy π (including the optimal policy π^*), function approximation can represent the true Q^π exactly. Which of the following statements are true?

☐ When $N = 1$, $Q_{\phi_{k+1}}$ is an unbiased estimate of the Q-function of the last policy Q^{π_k}.

☐ When $N = 1$ and in the limit as $B \to \infty$, $k \to \infty$, Q_{ϕ_k} converges to Q^*.

☐ When $N > 1$ (but finite) and in the limit as $B \to \infty$, $k \to \infty$, Q_{ϕ_k} converges to Q^*.

☐ When $N \to \infty$ and in the limit as $B \to \infty$, $k \to \infty$, Q_{ϕ_k} converges to Q^*.

1.5 Multistep Importance Sampling (5 points)

We can use importance sampling to make the N-step update work off-policy with trajectories drawn from an arbitrary policy. Rewrite (2) to correctly approximate a Q_{ϕ_k} that improves upon π when it is trained on data $\mathcal{D}$ consisting of B rollouts of some other policy $\pi'(\mathbf{a}_t \mid \mathbf{s}_t)$.

我们作出以下额外假设：

- 状态和动作空间是有限的。
- 每个批次至少包含在每个状态中对每个动作采取的一条经验。
- 在表格设置中， Q_{ϕ_k} 可以表示任意函数，即 $\{Q_{\phi_k} : \phi \in \Phi\} = \mathbb{R}^{S \times A}$。

当在算法1的第3行用 B 条新轨迹更新缓冲区 $\mathcal{D}_k$ 时，我们说：

- 在进行在策略内学习时， $\mathcal{D}_k$ 被设置为只包含来自 π 的 B 条新采样轨迹（因此 $|\mathcal{D}_k| = B$）。因此，我们只在来自当前策略的采样轨迹上进行训练。
- 在进行离策略学习时，我们使用一个固定的数据集 $\mathcal{D}_k = \mathcal{D}$ ，该数据集包含来自另一策略 π'的 B 条轨迹。

指出在下面情况下，哪些陈述**I-III** 始终成立。无需给出理由。

1. $N = 1$和... **I. II. III.**(a) 表格形式下的在策略情况 ☐ ☐ ☐(b) 表格形式下的离策略情况 ☐ ☐ ☐

2. $N > 1$和……
   (a) 表格设置下的在策略 ☐ ☐ ☐
   (b) 表格设置下的离策略 ☐ ☐ ☐
3. 当 $N \to \infty$ 趋于极限时（不进行自举）……
   (a) 表格设置下的在策略 ☐ ☐ ☐(b) 表格设置下的离策略 ☐ ☐ ☐

1.3 Q估计的方差（2 分）

在无限次迭代 k 的极限下，对于固定数据集大小 B ，你预计三种情况中的哪一种（$N = 1$、 $N > 1$、 $N \to \infty$）会对 Q 的估计具有最高方差？哪一种方差最低？

方差最高：

☐ $N = 1$
☐ $N > 1$
☐ $N \to \infty$

方差最低：

☐ $N = 1$
☐ $N > 1$
☐ $N \to \infty$

1.4 函数逼近（2 分）

现在假设我们想用函数逼近来表示 Q ，而不是用表格表示。假设对于任意确定性策略 π （包括最优策略 π^*），函数逼近都能精确表示真实的 Q^π 。下列哪些陈述为真？

☐ 当 $N = 1$， $Q_{\phi_{k+1}}$ 是对上一个策略 Q^{π_k} 的 Q 函数的无偏估计时。☐ 当 $N = 1$ 并且在 $B \to \infty, k \to \infty$ 的极限下， Q_{ϕ_k} 收敛到 Q^*。☐ 当 $N > 1$ （但为有限）并且在 $B \to \infty, k \to \infty$ 的极限下， Q_{ϕ_k} 收敛到 Q^*。☐ 当 $N \to \infty$ 并且在 $B \to \infty, k \to \infty$ 的极限下， Q_{ϕ_k} 收敛到 Q^*。

1.5 多步重要性采样（5 分）

我们可以使用重要性采样，使 N步更新在从任意策略采样得到的轨迹上也能进行离策略更新。重写(2)，以便在用由另一个策略 π'（$\mathbf{a}_t \mid \mathbf{s}_t$）生成、由 B 次轨迹组成的数据$\mathcal{D}$ 上训练时，正确地近似一个能够改进 π 的 Q_{ϕ_k}。

Do we need to change (2) when $N = 1$? What about as $N \to \infty$?

You may assume that π' always assigns positive mass to each action. [Hint: re-weight each term in the sum using a ratio of likelihoods from the policies π and π'.]

2 Deep Q-Learning

2.1 Introduction

Part 1 of this assignment requires you to implement and evaluate Q-learning for playing Atari games. The Q-learning algorithm was covered in lecture, and you will be provided with starter code. This assignment will be faster to run on a GPU, though it is possible to complete on a CPU as well. Note that we use convolutional neural network architectures in this assignment. Therefore, we recommend using the Colab option if you do not have a GPU available to you. Please start early!

2.2 File overview

The starter code for this assignment can be found at

`https://github.com/berkeleydeeprlcourse/homework_fall2023/tree/main/hw3`

You will implement a DQN agent in `cs285/agents/dqn_agent.py` and `cs285/scripts/run_hw3_dqn.py`. In addition to those two files, you should start by reading the following files thoroughly:

- `cs285/env_configs/dqn_basic.py`: builds networks and generates configuration for the basic DQN problems (cartpole, lunar lander).
- `cs285/env_configs/dqn_atari.py`: builds networks and generates configuration for the Atari DQN problems.
- `cs285/infrastructure/replay_buffer.py`: implementation of replay buffer. You don't need to know how the memory efficient replay buffer works, but you should try to understand what each method does (particularly the difference between `insert`, which is called after a frame, and `on_reset`, which inserts the first observation from a trajectory) and how it differs from the regular replay buffer.
- `cs285/infrastructure/atari_wrappers.py`: contains some wrappers specific to the Atari environments. These wrappers can be key to getting challenging Atari environments to work!

There are two new package requirements (`gym[atari]` and `pip install gym[accept-rom-license]`) beyond what was used in the first two assignments; make sure to install these with `pip install -r requirements.txt` if you're re-using your Python environment from last assignment.

2.3 Implementation

The first phase of the assignment is to implement a working version of Q-learning, with some extra bells and whistles like double DQN. Our code will work with both state-based environments, where our input is a low-dimensional list of numbers (like Cartpole), but we'll also support learning directly from pixels!

In addition to the double Q-learning trick (which you'll implement later), we have a few other tricks implemented to stabilize performance. You don't have to do anything to enable these, but you should look at the implementations and think about why they work.

- **Exploration scheduling for ϵ-greedy actor.** This starts ϵ at a high value, close to random sampling, and decays it to a small value during training.
- **Learning rate scsheduling.** Decay the learning rate from a high initial value to a lower value at the end of training.
- **Gradient clipping.** If the gradient norm is larger than a threshold, scale the gradients down so that the norm is equal to the threshold.

当 $N = 1$时，我们需要更改(2)吗？作为 $N \to \infty$时又如何?

你可以假设 π' 始终对每个动作分配正概率。 [提示：使用策略 π 和 π'的似然比来对和中的每一项重新加权。]

2 深度Q学习

2.1 引言

本次作业的第 1 部分要求你实现并评估用于玩 Atari 游戏的 Q-learning。Q-learning 算法在课堂上已有讲解，并且会提供起始代码。本作业在 GPU 上运行会更快，尽管在 CPU 上也可以完成。注意本次作业使用卷积神经网络架构。因此，如果你没有可用的 GPU，我们建议使用 Colab 选项。请尽早开始!

2.2 文件概览

本次作业的起始代码可在以下位置找到:

https://github.com/berkeleydeeprlcourse/homework_fall2023/tree/main/hw 3

你需要在 cs285/agents/dqn_agent.py 和 cs285/scripts/run_hw3_dqn.py 中实现一个 DQN 智能体。除了这两个文件外，你还应首先仔细阅读以下文件:

- cs285/env_configs/dqn_basic.py：为基本的 DQN 问题（cartpole, lunar lander）构建网络并生成配置。
- cs285/env_configs/dqn_atari.py: 为 Atari DQN 问题构建网络并生成配置。
- cs285/infrastructure/replay_buffer.py：重放缓冲区的实现。你无需了解内存高效重放缓冲区的工作原理，但应尝试理解每个方法的作用（特别是 insert —— 在一帧之后调用，与 on_reset —— 在一个轨迹开始时插入第一条观测之间的区别）以及它与常规重放缓冲区的不同之处。
- cs285/infrastructure/atari_wrappers.py：包含一些针对 Atari 环境的包装器。这些包装器可能是让具有挑战性的 Atari 环境正常运行的关键!

除前两次作业使用的包之外，还有两个新的包依赖（gym[atari] 和 pip install gym[accept-rom-license]）；如果你要重用上次作业的 Python 环境，请务必通过 pip install -r requirements.txt 安装这些包。

2.3 实现

本作业的第一阶段是实现一个可运行的 Q-learning 版本，并加入一些额外功能，比如 double DQN。我们的代码既能用于基于状态的环境（输入为低维数值列表，例如 Cartpole），也支持直接从像素学习!

除了双重 Q-学习技巧（你稍后将实现），我们还实现了一些其他技巧来稳定性能。你无需做任何事情来启用这些技巧，但应查看它们的实现并思考为何有效。

- **用于探索调度 ϵ-贪婪 actor**。这会使 ϵ 以接近随机采样的较高值开始，并在训练过程中衰减到较小的值。
- **学习率调度**。将学习率从较高的初始值衰减到训练结束时的较低值。
- **梯度裁剪**。如果梯度范数大于某个阈值，则缩放梯度，使范数等于该阈值。

- **Atari wrappers.**
  - **Frame-skip.** Keep the same constant action for 4 steps.
  - **Frame-stack.** Stack the last 4 frames to use as the input.
  - **Grayscale.** Use grayscale images.

2.4 Basic Q-Learning

Implement the basic DQN algorithm. You'll implement an update for the Q-network, a target network, and

What you'll need to do:

- Implement a DQN critic update in `update_critic` by filling in the unimplemented sections (marked with TODO(student)).
- Implement ϵ-greedy sampling in `get_action`
- Implement the TODOs in `run_hw3_dqn.py`.

  Hint: A trajectory can end (`done=True`) in two ways: the actual end of the trajectory (usually triggered by catastrophic failure, like crashing), or *truncation*, where the trajectory doesn't actually end but we stop simulation for some reason (commonly, we truncate trajectories at some maximum episode length). In this latter case, you should still reset the environment, but the `done` flag for TD-updates (stored in the replay buffer) should be false.
- Call all of the required updates, and update the target critic if necessary, in `update`.

Testing this section:

- Debug your DQN implementation on `CartPole-v1` with `experiments/dqn/cartpole.yaml`. It should reach reward of nearly 500 within a few thousand steps.

Deliverables:

- Submit your logs of `CartPole-v1`, and a plot with environment steps on the x-axis and eval return on the y-axis.
- Run DQN with three different seeds on `LunarLander-v2`:

```
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander.yaml --seed 1
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander.yaml --seed 2
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander.yaml --seed 3
```

  Your code may not reach high return (200) on Lunar Lander yet; this is okay! Your returns may go up for a while and then collapse in some or all of the seeds.
- Run DQN on `CartPole-v1`, but change the `learning rate` to 0.05 (you can change this in the YAML config file). What happens to (a) the predicted Q-values, and (b) the critic error? Can you relate this to any topics from class or the analysis section of this homework?

2.5 Double Q-Learning

Let's try to stabilize learning. The double-Q trick avoids overestimation bias in the critic update by using two different networks to *select* the next action a' and to *estimate* its value:

$$a' = \arg\max_{a'} Q_\phi(s', a')$$

$$Q_{\text{target}} = r + \gamma(1 - d_t)Q_{\phi'}(s', a').$$

In our case, we'll keep using the target network $Q_{\phi'}$ to estimate the action's value, but we'll select the action using Q_ϕ (the online Q network).

- **Atari 封装器**。

  – 跳帧。在 4 个时间步内保持相同动作。**– 帧堆叠**。将最近的 4 帧堆叠作为输入。**– 灰度化**。使用灰度图像。

2.4 基本 Q 学习

实现基本的 DQN 算法。你将为 Q-网络、目标网络实现一次更新，以及

你需要完成的任务:

- 在 update_critic 中实现 DQN 的 critic 更新，填写未实现的部分（标记为 TODO(student)）。
- 在 get_action 中实现 ϵ-贪心采样
- 在 run_hw3_dqn.py 中实现标记为 TODO 的部分。

  提示：一个轨迹可以通过两种方式结束（done=True）：轨迹的实际结束（通常由灾难性失败触发，例如撞车），或者截断，此时轨迹实际上并未结束，但我们因某种原因停止仿真（通常在达到最大回合长度时截断轨迹）。在后一种情况下，你仍应重置环境，但用于 TD 更新的 done 标志（存储在重放缓冲区中）应为 false。
- 在 update 中调用所有必需的更新，并在必要时更新目标 critic。

测试本节:

- 调试你在 CartPole-v1 上的 DQN 实现，使用 experiments/dqn/cartpole.yaml。它应在几千步内达到接近 500 的奖励。

交付物:

- 提交你的 CartPole-v1 日志，以及一张图，横轴为 x（环境步数），纵轴为 y（评估回报）。
- 在 LunarLander-v2 上用三个不同的随机种子运行 DQN：

  python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander.yaml --seed 1 python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander.yaml --seed 2 python

  你的代码可能还达不到 Lunar Lander 的高回报（200）；这没关系！你的回报可能会在一段时间内上升，然后在某些或所有随机种子中崩溃。
- 在 CartPole-v1 上运行 DQN，但将学习率改为 0.05（你可以在 YAML 配置文件中更改）。（a）预测的 Q 值会发生什么变化，（b）评论家误差会发生什么变化？你能把这与课堂上的任何主题或本次作业的分析部分联系起来吗?

2.5 双重 Q 学习

让我们尝试稳定学习过程。双重 Q 技巧通过使用两个不同的网络来避免评论家更新中的高估偏差：一个用于选择下一个动作 a'，另一个用于估计其值：

$$a' = \arg\max_{a'} Q_\phi(s', a')$$

$$Q_{\text{target}} = r + \gamma(1 - d_t)Q_{\phi'}(s', a').$$

在我们的情况下，我们会继续使用目标网络 $Q_{\phi'}$ 来估计动作的价值，但在选择动作时我们会使用 Q_ϕ（即在线的 Q 网络）。

Implement this functionality in `dqn_agent.py`.

Deliverables:

- Run three more seeds of the lunar lander problem:

```
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander_doubleq.yaml --
    seed 1
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander_doubleq.yaml --
    seed 2
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander_doubleq.yaml --
    seed 3
```

  You should expect a return of **200** by the end of training, and it should be fairly stable compared to your policy gradient methods from HW2.

  Plot returns from these three seeds in red, and the "vanilla" DQN results in blue, on the same set of axes. Compare the two, and describe in your own words what might cause this difference.

- Run your DQN implementation on the `MsPacman-v0` problem. Our default configuration will use double-Q learning by default. You are welcome to tune hyperparameters to get it to work better, but the default parameters should work (so if they don't, you likely have a bug in your implementation). Your implementation should receive a score of around **1500** by the end of training (1 million steps. **This problem will take about 3 hours with a GPU, or 6 hours without, so start early!**

```
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/mspacman.yaml
```

- Plot the average training return (`train_return`) and eval return (`eval_return`) on the same axes. You may notice that they look very different early in training! Explain the difference.

2.6 Experimenting with Hyperparameters

Now let's analyze the sensitivity of Q-learning to hyperparameters. Choose one hyperparameter of your choice and run at least three other settings of this hyperparameter, in addition to the one used in Question 1, and plot all four values on the same graph. Your choice what you experiment with, but you should explain why you chose this hyperparameter in the caption. Create four config files in `experiments/dqn/hyperparameters`, and look in `cs285/env_configs/basic_dqn_config.py` to see which hyperparameters you're able to change. You can use any of the base YAML files as a reference.

Hyperparameter options could include:

- Learning rate
- Network architecture
- Exploration schedule (or, if you'd like, you can implement an alternative to ϵ-greedy)

3 Continuous Actions with Actor-Critic

DQN is great for discrete action spaces. However, it requires you to be able to calculate $\max_a Q(s, a)$ in closed form. Doing this is trivial for discrete action spaces (when you can just check which of the n actions has the highest Q-value), but in continuous action spaces this is potentially a complex nonlinear optimization problem.

Actor-critic methods get around this by learning two networks: a Q-function, like DQN, and an explicit policy π that is trained to maximize $\mathbb{E}_{a\sim\pi(a|s)}Q(s, a)$.

All parts in this section are run with the following command:

```
python cs285/scripts/run_hw3_sac.py -cfg experiments/sac/<CONFIG>.yaml
```


在 dqn_agent.py 中实现此功能。

交付物:

- 为月球着陆器问题再运行另外三组随机种子:

```
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander_doubleq.yaml --
seed 1 python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/lunarlander_
doubleq.yaml -- seed 2 python cs285/scripts/run_hw3_dqn.py -cfg
experiments/dqn/lunarlander_doubleq.yaml -- seed 3
```

  你应该期望回报为 **200** 在训练结束时，并且与你在作业2中的策略梯度方法相比，它应该相当稳定。

  在同一组坐标轴上，用红色绘制这三个随机种子的回报，用蓝色绘制"原始"DQN的结果。比较二者，并用你自己的话描述可能导致这种差异的原因。

- Run your DQN implementation on the `MsPacman-v0` problem. Our default configuration will use double-Q learning by default. You are welcome to tune hyperparameters to get it to work better, but the default parameters should work (so if they don't, you likely have a bug in your implementation). Your implementation should receive a score of around **1500** by the end of training (1 million steps. **This problem will take about 3 hours with a GPU, or 6 hours without, so start early!**

```
python cs285/scripts/run_hw3_dqn.py -cfg experiments/dqn/mspacman.yaml
```

- 在同一坐标轴上绘制平均训练回报 (训练_回报) 和评估回报 (评估_回报)。你可能会注意到它们在训练初期看起来非常不同！解释这种差异。

2.6 超参数实验

现在让我们分析 Q-learning 对超参数的敏感性。选择一个你想要的超参数，除了在第1题中使用的设置外，还至少运行该超参数的另外三种设置，并将这四个值绘制在同一张图上。你可以自行选择要实验的内容，但应在图注中解释你选择此超参数的原因。在 experiments/dqn/hyperparameters 中创建四个配置文件，并查看 cs285/env_configs/basic_dqn_config.py 以了解哪些超参数可以更改。你可以使用任意基础 YAML 文件作为参考。

超参数选项可以包括:

- 学习率
- 网络架构
- 探索计划（或者，如果你愿意，你可以实现一种 ϵ-贪婪的替代方法）

3 连续动作与 Actor-Critic

DQN 在离散动作空间中表现出色。然而，它要求你能够以解析形式计算 $\max_a Q(s, a)$。对于离散动作空间来说这很简单（当你可以直接检查哪个 n 动作具有最高的 Q值），但在连续动作空间中，这可能是一个复杂的非线性优化问题。

Actor-critic 方法通过学习两个网络来解决这个问题：一个像 DQN 一样的 Q-函数，以及一个显式策略π，该策略被训练为最大化 $\mathbb{E}_{a\sim\pi(a|s)}Q(s, a)$。
本节中所有部分均使用以下命令运行:

```
python cs285/scripts/run_hw3_sac.py -cfg experiments/sac/<CONFIG>.yaml
```

3.1 Implementation

First, you'll need to take a look at the following files:

- `cs285/scripts/run_hw3_sac.py` - the main training loop for your SAC implementation.
- `cs285/agents/soft_actor_critic.py` - the structure for the SAC learner you'll implement.

You may also find the following files useful:

- `cs285/networks/state_action_critic.py` - a simple MLP-based $Q(s, a)$ network. Note that unlike the DQN critic, which maps states to an array of Q-value, one per action, this critic maps one (s, a) pair to a single Q-value.
- `cs285/env_configs/sac_config.py` - base configuration (and list of hyperparameters).
- `experiments/sac/*.yaml` - configuration files for the experiments.

You'll primarily be implementing your code in `cs285/agents/soft_actor_critic.py`.

Before implementing SAC:

- Fill in all of the `TODO`s in `cs285/scripts/run_hw3_sac.py`. This should look pretty similar to your DQN run script, as both are off-policy methods!

3.1.1 Bootstrapping

As in DQN, we train our critic by "bootstrapping" from a target critic. Using the tuple $(s_t, a_t, r_t, s_{t+1}, d_t)$ (where d_t is the flag for whether the trajectory terminates after this transition), we write:

$$y \leftarrow r_t + \gamma(1 - d_t)Q_\phi(s_{t+1}, a_{t+1}), a \sim \pi(a_{t+1}|s_{t+1})$$

$$\min_\phi (Q_\phi(s_t, a_t) - y)^2$$

In practice, we stabilize learning by using a separate *target network* $Q_{\phi'}$. There are two common strategies for updating the target network:

- *Hard update* (like we implemented in DQN), where every K steps we set $\phi' \leftarrow \phi$.
- *Soft update*, where ϕ' is continually updated towards ϕ with *Polyak averaging* (exponential moving average). After each step, we perform the following operation:

$$\phi' \leftarrow \phi' + \tau(\phi - \phi')$$

What you'll need to do (in `cs285/agents/soft_actor_critic.py`):

- Implement the bootstrapped critic update in the `update_critic` method.
- Update the critic for `num_critic_updates` in the `update` method.
- Implement soft and hard target network updates, depending on the configuration, in `update`.

Testing this section:

- Train an agent on `Pendulum-v1` with the sample configuration `experiments/sac/sanity_pendulum.yaml`. It shouldn't get high reward yet (you're not training an actor), but the Q-values should stabilize at some large negative number. The "do-nothing" reward for this environment is about -10 per step; you can use that together with the discount factor γ to calculate (approximately) what Q should be. If the Q-values go to minus infinity or stay close to zero, you probably have a bug.

Deliverables: None, once the critic is training as expected you can move on to the next section!

3.1 实现

首先，您需要查看以下文件：

- cs285/scripts/run_hw3_sac.py - 你实现的 SAC 的主要训练循环。
- cs285/agents/soft_actor_critic.py - 你将实现的 SAC 学习器的结构 nt.

你可能还会发现以下文件有用：

- cs285/networks/state_action_critic.py - 一个简单的基于 MLP 的 $Q(s, a)$ 网络。注意，与 DQN 的评估器不同，DQN 将状态映射为每个动作对应的 Q 值数组，而该评估器将一对 (s, a) 映射为单个 Q 值。
- cs285/env_configs/sac_config.py - 基本配置（以及超参数列表）。
- experiments/sac/*.yaml - 实验的配置文件。

Y你将主要在 cs285/agents/soft_actor_critic.py 中实现你的代码。

在实现 SAC 之前：

- 在 cs285/scripts/run_hw3_sac.py 中完成所有 TODO。这应该与您的 DQN 运行脚本非常相似，因为两者都是离策略方法！

3.1.1 自举

正如在 DQN 中，我们通过从目标评论家“自举”来训练我们的评论家。使用元组 $(s_t, a_t, r_t, s_{t+1}, d_t)$ (其中 d_t 是表示该轨迹在此转移后是否终止的标志), 我们写道：

$$y \leftarrow r_t + \gamma(1 - d_t)Q_\phi(s_{t+1}, a_{t+1}), a \sim \pi(a_{t+1}|s_{t+1})$$

$$\min_\phi (Q_\phi(s_t, a_t) - y)^2$$

在实践中，我们通过使用一个独立的目标网络 $Q_{\phi'}$. 更新目标网络有两种常见策略：

- 硬更新 (就像我们在 DQN 中实现的那样), 其中每 K 步我们将 $\phi' \leftarrow \phi$ 设为.
- 软更新，其中 ϕ' 会持续向 ϕ 更新，使用*Polyak* 平均 （指数移动平均）。每一步之后，我们执行以下操作：

$$\phi' \leftarrow \phi' + \tau(\phi - \phi')$$

你需要完成的内容 (位于 cs285/agents/soft_actor_critic.py)：

- 在 update_critic 方法中实现自举（bootstrapped）评论器更新。
- 在 update 方法中按 num_critic_updates 对 critic 进行更新。
- 在 update 中根据配置实现软（soft）和硬（hard）目标网络更新。

测试本节：

- 在 Pendulum-v1 上使用示例配置 experiments/sac/sanity_pendulum.yaml 训练智能体。 . 它目前不应该获得高回报（你并未在训练 actor），但 Q 值应稳定在某个较大的负数附近。该环境的“无动作”回报约为每步 -10；你可以将其与折扣因子 γ 一起用于（大致）计算 Q 应该是多少。如果 Q 值趋向负无穷或接近于零，说明很可能存在错误。

可交付成果：无，一旦评论家按预期开始训练，你就可以进入下一节！

3.1.2 Entropy Bonus and Soft Actor-Critic

In DQN, we used an ϵ-greedy strategy to decide which action to take at a given time. In continuous spaces, we have several options for generating exploration noise.

One of the most common is providing an *entropy bonus* to encourage the actor to have high entropy (i.e. to be "more random"), scaled by a "temperature" coefficient β. For example, in the REPARAMETRIZE case:

$$\mathcal{L}_\pi = Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon) + \beta\mathcal{H}(\pi(a|s)).$$

Where entropy is defined as $\mathcal{H}(\pi(a|s)) = \mathbb{E}_{a\sim\pi}[-\log \pi(a|s)]$. To make sure entropy is also factored into the Q-function, we should also account for it in our target values:

$$y \leftarrow r_t + \gamma(1-d_t)\left[Q_\phi(s_{t+1}, a_{t+1}) + \beta\mathcal{H}(\pi(a_{t+1}|s_{t+1}))\right]$$

When balanced against the "maximize Q" terms, this results in behavior where the actor will choose more random actions when it is unsure of what action to take. Feel free to read more in the SAC paper: `https://arxiv.org/abs/1801.01290`.

Note that maximizing entropy $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta]$ requires differentiating *through* the sampling distribution. We can do this via the "reparametrization trick" from lecture - if you'd like a refresher, skip to the section on REPARAMETRIZE.

What you'll need to do (in `cs285/agents/soft_actor_critic.py`):

- Implement `entropy()` to calculate the approximate entropy of an actor distribution.
- Add the entropy term to the target critic values in `update_critic()` and the actor loss in `update_actor()`.

Testing this section:

- The code should be logging `entropy` during the critic updates. If you run `sanity_pendulum.yaml` from before, it should achieve (close to) the maximum possible entropy for a 1-dimensional action space. Entropy is maximized by a uniform distribution:

$$\mathcal{H}(\mathcal{U}[-1,1]) = \mathbb{E}[-\log p(x)] = -\log\frac{1}{2} = \log 2 \approx 0.69$$

  Because currently our actor loss **only** consists of the entropy bonus (we haven't implemented anything to maximize rewards yet), the entropy should increase until it arrives at roughly this level.

  If your logged entropy is higher than this, or significantly lower, you have a bug.

3.1.3 Actor with REINFORCE

We can use the same REINFORCE gradient estimator that we used in our policy gradients algorithms to update our actor in actor-critic algorithms! We want to compute:

$$\nabla_\theta \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(a|s)}\left[Q(s,a)\right]$$

To do this using REINFORCE, we can use the policy gradient:

$$\mathbb{E}_{s\sim\mathcal{D}, a\sim\pi(a|s)}\left[\nabla_\theta \log(\pi_\theta(a|s))Q_\phi(s,a)\right]$$

Note that the actions a are sampled from π_θ, and we do **not** require real data samples. This means that to reduce variance we can just sample more actions from π for any given state! You'll implement this in your code using the `num_actor_samples` parameter.

What you'll need to do (in `cs285/agents/soft_actor_critic.py`):

- Implement the REINFORCE gradient estimator in the `actor_loss_reinforce` method.
- Update the actor in `update`.

3.1.2 熵奖励与软演员-评论家

在 DQN 中，我们使用了 ϵ-贪婪策略来决定在给定时刻采取哪个动作。在连续空间中，我们有多种生成探索噪声的选项。

最常见的方法之一是提供一个熵奖励，以鼓励策略（actor）具有较高的熵（即“更随机”），并按一个“温度”系数 β 缩放。例如，在 REPARAMETRIZE 情况下：

$$\mathcal{L}_\pi = Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon) + \beta\mathcal{H}(\pi(a|s)).$$

其中熵定义为 $\mathcal{H}(\pi(a|s)) = \mathbb{E}_{a\sim\pi}[-\log \pi(a|s)]$。为了确保熵也被纳入到$Q$-函数中，我们还应在目标值中考虑它：

$$y \leftarrow r_t + \gamma(1-d_t)\left[Q_\phi(s_{t+1}, a_{t+1}) + \beta\mathcal{H}(\pi(a_{t+1}|s_{t+1}))\right]$$

当与“maximize Q”项相权衡时，这会导致一种行为：当策略（actor）不确定采取何种动作时，会选择更随机的动作。想了解更多可参阅 SAC 论文：https: //arxiv.org/abs/1801.01290。

注意，最大化熵 $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta]$ 需要通过对采样分布进行微分。我们可以使用讲座中的“重参数化技巧”来实现——如果你想复习，请跳到 REPARAMETRIZE 一节。

你需要完成的事项(在 cs285/agents/soft_actor_critic.py):

- 实现 entropy() 来计算 actor 分布的近似熵 .
- 将熵项添加到 update_critic() 中的目标 critic 值，以及添加到 update_actor() 中的 actor 损失。

测试本节:

- 代码应在 critic 更新期间记录熵。如果你运行之前的 sanity_pendulum.yaml，它应能达到（或接近）一维动作空间的最大可能熵。熵由均匀分布最大化：

$$\mathcal{H}(\mathcal{U}[-1,1]) = \mathbb{E}[-\log p(x)] = -\log\frac{1}{2} = \log 2 \approx 0.69$$

  因为目前我们的 actor 损失 **仅** 由熵奖励组成（我们还未实现任何用于最大化回报的内容），因此熵应当增加直到大致达到这一水平。

  如果你记录的熵高于此值，或显著低于此值，那就是有错误。

3.1.3 使用 REINFORCE 的 Actor

我们可以在 actor-critic 算法中使用与我们在策略梯度算法中使用的相同 REINFORCE 梯度估计器来更新我们的 actor！我们想要计算：

$$\nabla_\theta \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(a|s)}\left[Q(s,a)\right]$$

为此，使用 REINFORCE 时我们可以采用策略梯度：

$$\mathbb{E}_{s\sim\mathcal{D}, a\sim\pi(a|s)}\left[\nabla_\theta \log(\pi_\theta(a|s))Q_\phi(s,a)\right]$$

注意，动作 a 是从 π_θ 中采样，并且我们**不** 需要真实数据样本。这意味着，为了减少方差，对于任意给定状态我们可以只是从 π 中多采样动作！你将通过 num_actor_samples 参数在代码中实现这一点。

你需要完成的内容 (在 cs285/agents/soft_actor_critic.py 中):

- I在 actor_loss_reinforce 方法中实现 REINFORCE 梯度估计器 .
- 在 update 中更新 actor。

Testing this section:

- Train an agent on `InvertedPendulum-v4` using `sanity_invertedpendulum_reinforce.yaml`. You should achieve reward close to 1000, which corresponds to staying upright for all time steps.

Deliverables

- Train an agent on `HalfCheetah-v4` using the provided config (`halfcheetah_reinforce1.yaml`). Note that this configuration uses only one sampled action per training example.
- Train another agent with `halfcheetah_reinforce_10.yaml`. This configuration takes many samples from the actor for computing the REINFORCE gradient (we'll call this REINFORCE-10, and the single-sample version REINFORCE-1). Plot the results (evaluation return over time) on the same axes as the single-sample REINFORCE. Compare and explain your results.

3.1.4 Actor with REPARAMETRIZE

REINFORCE works quite well with many samples, but particularly in high-dimensional action spaces, it starts to require a lot of samples to give low variance. We can improve this by using the reparametrized gradient. Parametrize π_θ as $\mu_\theta(s) + \sigma_\theta(s)\epsilon$, where ϵ is normally distributed. Then we can write:

$$\nabla_\theta \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(a|s)}\left[Q(s,a)\right] = \nabla_\theta \mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}}\left[Q(s,\mu_\theta(s)+\sigma_\theta(s)\epsilon))\right] = \mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}}\left[\nabla_\theta Q(s,\mu_\theta(s)+\sigma_\theta(s)\epsilon))\right]$$

This gradient estimator often gives a much lower variance, so it can be used with few samples (in practice, just using a single sample tends to work very well).

Hint: you can use `.rsample()` to get a *reparametrized* sample from a distribution in PyTorch.

What you'll need to do:

- Implement `actor_loss_reparametrize()`. Be careful to use the reparametrization trick for sampling!

Testing this section:

- Make sure you can solve `InvertedPendulum-v4` (use `sanity_invertedpendulum_reparametrize.yaml`) and achieve similar reward to the REINFORCE case.

Deliverables:

- Train (once again) on `HalfCheetah-v4` with `halfcheetah_reparametrize.yaml`. Plot results for all three gradient estimators (REINFORCE-1, REINFORCE-10 samples, and REPARAMETRIZE) on the same set of axes, with number of environment steps on the x-axis and evaluation return on the y-axis.
- Train an agent for the `Humanoid-v4` environment with `humanoid_sac.yaml` and plot results.

3.1.5 Stabilizing Target Values

As in DQN, the target Q with a single critic exhibits *overestimation bias*! There are a few commonly-used strategies to combat this:

- Double-Q: learn two critics Q_{ϕ_A}, Q_{ϕ_B}, and keep two target networks $Q_{\phi'_A}, Q_{\phi'_B}$. Then, use $Q_{\phi'_A}$ to compute target values for Q_{ϕ_B} and vice versa:

$$y_A = r + \gamma Q_{\phi'_B}(s', a')$$

$$y_B = r + \gamma Q_{\phi'_A}(s', a')$$

- *Clipped* double-Q: learn two critics Q_{ϕ_A}, Q_{ϕ_B} (and keep two target networks). Then, compute the target values as $\min(Q_{\phi'_A}, Q_{\phi'_B})$.

$$y_A = y_B = r + \gamma \min\left(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a')\right)$$

测试本节:

•训练一个代理 在 InvertedPendulum-v4 上使用 sanity_invertedpendulum_reinforce.yaml。你应该达到 r 奖励接近1000，这对应于在所有时间步保持直立。

提交内容

• 在 HalfCheetah-v4 上使用提供的配置（halfcheetah_reinforce1.yaml）训练一个智能体。注意该配置在每个训练样本中仅使用一次采样动作。

• 使用 halfcheetah_reinforce_10.yaml 再训练一个智能体。该配置从策略中采集多个样本以计算 REINFORCE 梯度（我们称之为 REINFORCE-10，而单样本版本称为 REINFORCE-1）。将结果（随时间变化的评估回报）与单样本 REINFORCE 在同一坐标轴上绘制。比较并解释你的结果。

3.1.4 带重参数化的 Actor

REINFORCE 在样本较多时效果很好，但在高维动作空间中，通常需要大量样本才能使方差变小。我们可以通过使用重参数化梯度来改进这一点。将 π_θ 参数化为 $\mu_\theta(s) + \sigma_\theta(s)\epsilon$，其中 ϵ 服从正态分布。然后我们可以写为：

$$\nabla_\theta \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(a|\ s)}\left[Q(s,a)\right] = \nabla_\theta \mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}}\left[Q(s,\mu_\theta(s)+\sigma_\theta(s)\epsilon))\right] = \mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}}\left[\nabla_\theta Q(s,\mu_\theta(s)+\sigma_\theta(s)\epsilon)\right]$$

这种梯度估计器通常会产生更低的方差，因此可以使用较少的样本（在实际中，仅使用单个样本通常就能很好地工作）。

提示: 你可以使用 .rsample() 从 PyTorch 的分布中获取一个可重参数化的样本。 h.

你需要做的内容:

• 实现 actor_损失_reparametrize()。注意在采样时使用重参数化技巧！

测试本节:

• 确保你能够解决 InvertedPendulum-v4（使用 sanity_invertedpendulum_reparametrize.yaml），并达到与 REINFORCE 情况相似的奖励。

交付物:

• 在 HalfCheetah-v4 上使用 halfcheetah_reparametrize.yaml 再次训练。将三种梯度估计器（REINFORCE-1、REINFORCE-10 样本和 REPARAMETRIZE）的结果绘制在同一坐标系中，横轴为环境步数（ x 轴），纵轴为评估回报（ y 轴）。

• 使用 humanoid_sac.yaml 在 Humanoid-v4 环境中训练一个智能体并绘制结果。

3.1.5 稳定目标值

与 DQN 相同，只有单一评论家的目标 Q 会表现出过度估计偏差！有几种常用策略可以应对：

• 双重-Q：学习两个评论家 Q_{ϕ_A}, Q_{ϕ_B}，并保留两个目标网络 $Q_{\phi'_A}, Q_{\phi'_B}$。然后，使用 $Q_{\phi'_A}$ 来为 Q_{ϕ_B} 计算目标值，反之亦然：

$$y_A = r + \gamma Q_{\phi'_B}(s', a')$$

$$y_B = r + \gamma Q_{\phi'_A}(s', a')$$

• 裁剪的 double-Q：学习两个评估器 Q_{ϕ_A}， Q_{ϕ_B} （并保留两个目标网络）。然后，将目标值计算为 $\min(Q_{\phi'_A}$， $Q_{\phi'_B})$.

$$y_A = y_B = r + \gamma \min\left(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a')\right)$$

- **(Optional, bonus)** *Ensembled* clipped double-Q: learn many critics (10 is common) and keep a target network for each. To compute target values, first run all the critics and sample two Q-values for each sample. Then, take the minimum (as in clipped double-Q). If you want to learn more about this, you can check out "Randomized Ensembled Double-Q": `https://arxiv.org/abs/2101.05982`.

Implement double-Q and clipped double-Q in the `q_backup_strategy` function in `soft_actor_critic.py`.

Deliverables:

- Run single-Q, double-Q, and clipped double-Q on `Hopper-v4` using the corresponding configuration files. Which one works best? Plot the logged `eval_return` from each of them as well as `q_values`. Discuss how these results relate to overestimation bias.
- Pick the best configuration (single-Q/double-Q/clipped double-Q, or REDQ if you implement it) and run it on `Humanoid-v4` using `humanoid.yaml` (edit the config to use the best option). You can truncate it after 500K environment steps. If you got results from the humanoid environment in the last homework, plot them together with environment steps on the x-axis and evaluation return on the y-axis. Otherwise, we will provide a humanoid log file that you can use for comparison. How do the off-policy and on-policy algorithms compare in terms of sample efficiency? *Note: if you'd like to run training to completion (5M steps), you should get a proper, walking humanoid! You can run with videos enabled by using `--nvid 1`. If you run with videos, you can strip videos from the logs for submission with this script.*

4 Submitting the code and experiment runs

In order to turn in your code and experiment logs, create a folder that contains the following:

- A folder named `data` with all the experiment runs from this assignment. **Do not change the names originally assigned to the folders, as specified by `exp_name` in the instructions. Video logging is disabled by default in the code, but if you turned it on for debugging, you will need to run those again with `--video_log_freq -1`, or else the file size will be too large for submission.**
- The `cs285` folder with all the `.py` files, with the same names and directory structure as the original homework repository (excluding the `data` folder). Also include any special instructions we need to run in order to produce each of your figures or tables (e.g. "run python myassignment.py -sec2q1" to generate the result for Section 2 Question 1) in the form of a README file.

As an example, the unzipped version of your submission should result in the following file structure. **Make sure that the submit.zip file is below 15MB and that they include the prefix `q1_`, `q2_`, `q3_`, etc.**

- **（可选，加分）** 集成的 裁剪的 double-Q：学习多个评估器（常见为10个），并为每个保留一个目标网络。要计算目标值，先运行所有评估器，然后为每个样本抽取两个 Q-值。接着取最小值（如裁剪的 double-Q）。如果你想了解更多，可以查看《Randomized Ensembled Double-Q》：https://arxiv.org/abs/2101.05982.

实现 double-Q 并在 soft_actor_critic.py 的 q_backup_strategy 函数中实现 clipped double-Q 。

交付物:

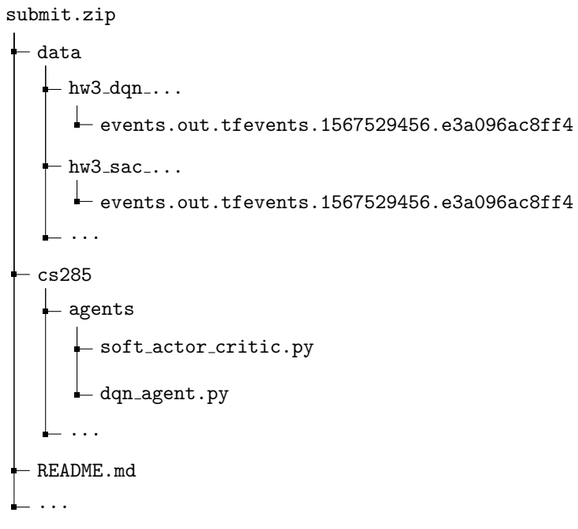
- Run single-Q, double-Q, and clipped double-Q on `Hopper-v4` using the corresponding configuration files. Which one works best? Plot the logged `eval_return` from each of them as well as `q_values`. Discuss how these results relate to overestimation bias.
- 选择最佳配置（single-Q/double-Q/clipped double-Q，或者如果你实现了 REDQ 则使用它）并使用 humanoid.yaml 在 Humanoid-v4 上运行（编辑配置以使用最佳选项）。你可以在 500K 环境步后截断。如果你在上一份作业中得到了 humanoid 环境的结果，请将它们与环境步在 x 轴上以及评估回报在 y 轴上一起绘制。否则，我们会提供一个 humanoid 日志文件供你比较。在样本效率方面，离策略（off-policy）算法与在策略（on-policy）算法相比如何？ 注意：如果你想将训练跑完（*5M* 步），你应该能得到一个真正会行走的 ***humanoid***！你可以通过使用 *`-nvid 1`*. 如果你启用了视频，你可以使用这个脚本从日志中剥离视频以用于提交。

4 提交代码和实验运行

为了 提交你的代码和实验日志，请创建一个包含以下内容的文件夹： :

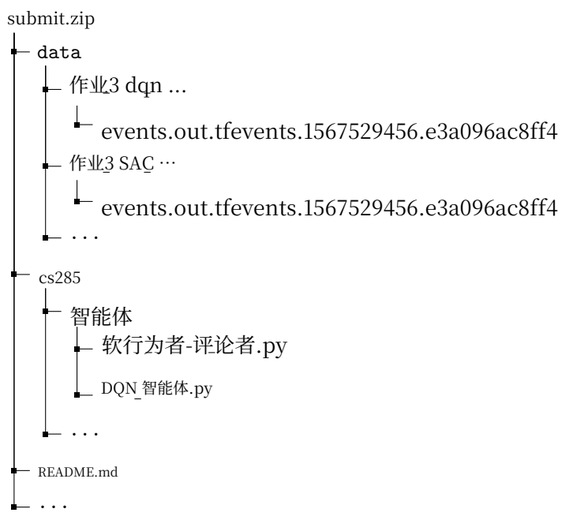
- 名为 `data` 且包含本次作业所有实验运行的文件夹。**不要更改最初分配给这些文件夹的名称，如说明中所指定的 `exp name` 。代码中默认禁用视频记录，但如果你为调试打开过视频记录，则需要再次使用 `--video log freq -1` 运行，否则文件大小将过大，无法提交。**
- 包含 cs285 文件夹及其所有 .py 文件，名称和目录结构与原始作业仓库相同（不包括 data 文件夹）。同时请以 README 文件的形式附上我们需要运行的任何特殊说明，以生成你的每张图或表（例如：“ run python myassignment.py -sec2q1” 来生成第 2 节第 1 问的结果）。

例如，解压后你的提交应生成以下文件结构。**确保 submit.zip 文件小于 15MB 并且它们包含前缀** q1 , q2 , q3 , **等**。



If you are a Mac user, **do not use the default "Compress" option to create the zip**. It creates artifacts that the autograder does not like. You may use `zip -vr submit.zip . -x "*.DS_Store"` from your terminal from within the top-level `cs285` directory.

Turn in your assignment on Gradescope. Upload the zip file with your code and log files to **HW3 Code**, and upload the PDF of your report to **HW3**.



如果你是 Mac 用户，**不要使用默认的"压缩"选项来创建 zip 文件**。它会生成自动评分器不喜欢的额外文件。你可以在顶层 cs285 目录下的终端中使用 zip -vr submit.zip . -x "*.DS Store"。

在 Gradescope 上提交你的作业。把包含代码和日志文件的 zip 上传到 **HW3 代码**，并把报告的 PDF 上传到 **HW3 报告**。

SAC-related questions. We wanted to address some of the common questions that have been asked regarding Question 6 of the HW. The full algorithm for SAC is summarized below, the equations listed in this paper will be helpful for you: `https://arxiv.org/pdf/1812.05905`. Some definitions that will be useful:

1. What is alpha and how to update it: Alpha is the entropy regularization coefficient denoting how much exploration to add to the policy. You should update based on Eq. 18 in Section 5 in the above paper as follows:
$$J(\alpha) = \mathop{\mathbb{E}}_{a_t \sim \pi_t} [-\alpha \log_{\pi_t}(a_t|s_t) - \alpha\bar{\mathcal{H}}].$$
2. Target entropy is the negative of the action space dimension that is used to update the alpha term.
3. SquashedNorm: This is a function that takes in mean and std as in previous homeworks, and will give you a distribution that you can sample your action from.
4. To update the critic, refer to how to update Q-function parameters in Equation 6 of the paper above as follows:
$$J_Q(\theta) = Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma(Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log(\pi_\phi(a_{t+1}, s_{t+1})))$$
5. To update the policy, follow Equation 18:
$$J(\alpha) = E_{a_t \sim \pi_t}[-\alpha \log \pi_t(a_t|s_t) - \alpha\bar{\mathcal{H}}]$$
6. You don't need to alter any parameters from the SAC run commands. The correct implementation should work with the provided default parameters.

与 SAC 相关的问题。我们想解答一些关于作业第6题的常见问题。下面总结了 SAC 的完整算法，论文中列出的公式会对你有帮助：https://arxiv.org/pdf/1812.05905。以下是一些有用的定义：

1. 什么是 alpha 以及如何更新：Alpha 是熵正则化系数，表示向策略中添加多少探索。你应该根据上述论文第5节的公式（Eq. 18）按如下方式更新：
$$J(\alpha) = \mathop{\mathbb{E}}_{a_t \sim \pi_t} [-\alpha \log_{\pi_t}(a_t|s_t) - \alpha\bar{\mathcal{H}}].$$
2. 目标熵等于动作空间维度的负值，用于更新 alpha 项。
3. SquashedNorm：这是一个函数，像之前的作业一样接受均值和标准差作为输入，并会为你提供一个可以从中采样动作的分布。
4. 要更新 critic（评论者），请参考上文论文中公式 6 关于如何更新 Q 函数参数的说明，如下：
$$J_Q(\theta) = Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma(Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log(\pi_\phi(a_{t+1}, s_{t+1})))$$
5. 要更新策略，请遵循公式 18：
$$J(\alpha) = E_{a_t \sim \pi_t}[-\alpha \log \pi_t(a_t|s_t) - \alpha\bar{\mathcal{H}}]$$
6. 你无需更改任何来自 SAC 运行命令的参数。正确的实现应能在提供的默认参数下正常工作。